

Technical Specification

Table of Contents

Table of Contents	1
Introduction	2
System Architecture	2
2.1 Architectural Overview	2
3.High-Level Design	5
4.Problems and Resolution	8
5. Software Testing	9
6.Installation Guide	12
6.1 Overview	12

Introduction

FinanceCoach is a cross-platform mobile budgeting application developed using the Flutter framework. The purpose of the system is to assist users, particularly university students and young adults, in managing their personal finances through expense tracking, budgeting, and savings goal monitoring.

The application allows users to create a monthly budget, record expenses under specific categories, track remaining funds, and monitor progress toward savings goals. The system provides real-time feedback by dynamically calculating totals and updating budget balances as expenses are entered. Data entered by the user is stored locally on the device to ensure persistence between sessions.

The application operates primarily as a standalone mobile system; however, it integrates with the OpenAI API to provide AI-powered financial guidance through a chatbot interface. When a user submits a query within the chat feature, the application sends a secure HTTP request to the OpenAI cloud service, which processes the request using a large language model and returns a generated response. This integration enhances the system by providing dynamic, context-aware financial advice.

System Architecture

2.1 Architectural Overview

The FinanceCoach application follows a layered architectural design to separate user interface logic, and data handling responsibilities. This separation improves maintenance, readability, and scalability of the system.

The architecture distributes responsibilities across clearly defined modules to ensure separation of concerns, maintainability, and scalability. Core budgeting features operate locally, while AI-based chat functionality communicates with a third-party cloud service.

The system consists of the following primary components:

1. Presentation Layer (User Interface)
2. Application Logic Layer
3. Data Storage Layer
4. External AI Service (OpenAI API)

2.2 Presentation Layer

The Presentation Layer is implemented using Flutter widgets and screens. This layer is responsible for:

- Rendering the user interface
- Capturing user input
- Displaying calculated financial data
- Displaying AI chatbot responses

The primary UI modules include:

- Dashboard / Home Screen
- Budget Management Screen
- Expense Entry Interface
- Goals Tracking Screen
- AI Chat Screen

Each screen interacts with the Application Logic Layer to retrieve, process, or update financial information.

2.3 Application Logic layer

The Application Logic Layer manages all core business functionality. This includes:

- Calculating total expenses
- Determining remaining budget
- Updating savings goal progress
- Validating user input fields
- Preparing chat prompts for AI requests

This layer ensures that financial calculations are performed independently from the user interface, improving modularity and enabling easier testing

2.4 Third-party and Reused Components

The following third-party components are used within the system:

- Flutter SDK – Cross-platform mobile development framework.
- Dart Programming Language – Core application logic.
- OpenAI API – Cloud-based AI service for chatbot functionality.
- Android and iOS SDKs – Platform deployment.

These components are integrated into the architecture to reduce development complexity and improve scalability.

2.5 External AI Service Integration

The system integrates with the OpenAI API to provide AI-powered financial guidance within the chat feature.

When a user submits a query in the chat interface:

1. The request is sent via HTTPS from the Flutter application.
2. The OpenAI cloud service processes the input using a language model.
3. A generated response is returned.
4. The response is displayed within the chat interface.

This external component is a third-party service and operates independently of the local budgeting logic.

Core budgeting functions remain fully operational even if the AI service is unavailable.

3.High-Level Design

3.1 Object Model

The Finance Coach application follows a client server architecture of a Flutter based mobile front-end and a backend service responsible for data. The design emphasises modularity and scalability along with separation of concerns so that the system is maintained and executable.

At a high level, the system is divided into the following major components:

Presentation Layer (Flutter Mobile App)

Application Logic Layer (Backend API)

Data Layer (Database/Storage)

External Services (If allowed)

The mobile application communicates with the backend via RESTful HTTP requests. The backend handles these requests by applying logic, interacting with the database, and returning structured responses to the client.

3.2 System Context

The application operates as a personal finance management tool that allows users to manage financial data and receive guidance on spending behaviour. The system environment includes:

Mobile users running the flutter application

Backend server hosting the Open AI API

Database for Storage

The mobile application is responsible solely for user interaction/presentation while all data management and essential processing are handled by the backend. This maintains data integrity and security.

3.3 Component Design

3.3.1 Flutter Frontend

The Flutter application provides the user interface and handles logic on the client's side. Its responsibility include:

- User Authentication Interface*
- Financial Inputs (Income/Expenses)*
- Displaying summaries*
- Sending HTTP requests to backend*
- Rendering responses from API*

The frontend in structures using Flutter widgets. This promotes maintainability and is easy to use.

3.3.2 Backend API

The backend service implements the core business logic of the application. Its responsibilities include:

- Processing client requests*
- Validating user input*
- Managing accounts*
- Interacting with database*
- Returning JSON responses*

The backend exposes REST endpoints which are used by the Flutter application. This separation allows the backend to be reused by other clients in the future if needed.

3.3.3 Data Storage Layer

Responsible for storage of application data such as:

- User accounts*
- Financial records*
- Budget Information*
- Historical data*

The backend communicates with the database through a data access layer. This ensures database operations are kept away from the rest of the system.

3.4 Data Flow

The data flow in the Finance Coach application is as follows:

The user performs an action in the Flutter application (eg. adds expense)
The frontend sends HTTP request to backend API
The backend validates the request and applies logic
The backend reads/writes data to database, which returns JSON response
The Flutter application updates the user interface.

This flow ensures clear separation between presentation, processing and storage.

3.5 Design Rationale

Several Design decisions were made for the application:

Client Server Architecture was chosen because it allows independent scalability.
Flutter was chosen to support cross-platform mobile development in the future.
Restful API design ensures interoperability and future extensibility.
Modular architecture improves testing.

This design supports future development and potential added functionality.

4.Problems and Resolution

4.1 Overview

During the design and implementation of our application, we encountered several technical and architectural challenges. Addressing these problems were needed to ensure system stability and usability.

4.2 Frontend to Backend Communication Issues

Our Problem

Early in application development, we experienced difficulties communicating the Flutter frontend reliably with the backend API:

Incorrect endpoint configuration
Timing out issues
JSON parsing errors

These problems prevented our application from fully functioning.

Our Resolution

Using Device IP configuration and standardising API endpoint definitions.
Implementing JSON response formats that were structured
Adding correct HTTP request handling in Flutter

After these changes were made, communication between frontend and backend was satisfactory.

4.3 Data Validation and Error Handling

Our Problem

Initial version of our backend would only provide limited validation on the data. This created risks such as:

Invalid financial entries
Potential crashes from bad requests
Poor user feedback

Our Resolution

We added validation checks to the API Layer to ensure that:

Required fields present
Numeric Values within valid ranges
Invalid requests return error messages

This improved our system and user experience.

4.4 Summary

The resolution of the frontend to backend communication issues and the introduction of better data validation helped improve the overall performance and stability of the application. By standardising API endpoints, configuring the correct device IP and improving HTTP handling, our application was able to communicate with the backend consistently.

In addition to this, implementing validation at the API layer ensures only valid financial data is processed/stored. This reduces the risk of crashes caused by malformed requests and provides clear feedback to users if an error occurs.

Together, these improvements improved the system exponentially, as well as the overall user experience.

5. Software Testing

5.1 Overview

Software testing was conducted throughout the development process to ensure correctness and reliability within the application. Testing focused on validating code functionality, verifying the frontend and backend relationship and ensuring invalid user input is handled. A layered testing approach was used. It included manual functional testing, API testing, unit/integration testing.

5.2 Approach and Sequence of Steps

Manual Functional Testing

The Flutter user interface was manually tested on Android Studio to ensure screens rendered correctly and navigation worked as expected.

Backend API Verification

API endpoints tested independently to verify correct status code, JSON response structures and behaviour.

Unit Testing

Minimum unit tests were implemented for key backend validation logic and frontend behaviours.

Integration Testing

The Flutter application was connected to the live backend to validate functionality, including submitting financial data and getting responses.

Regression Testing

Core Test Cases re-executed after fixes to ensure previously working functionality didn't break.

This structured process helped identify problems early and ensure we could fix them.

5.3 Tools and Frameworks

The following tools/frameworks were used:

Frontend (Flutter)

Flutter_test for unit and widget testing

Flutter emulator testing for manual validation

HTTP client mocking

Backend

API endpoint testing

Backend unit testing frameworks

These tools enabled automated and manual testing.

5.4 Functional Test Cases

Test CaseID	Feature	Pre-Condition	Steps	Exp. Result
1	Launch	App Installed	Open App	App loads, displays screen
2	Navigation	App running	Navigate between screens	Correct screen loads
3	Expense (Valid)	User on add expense screen	Enter valid data	Expense saved succesfully
4	Expense (Missing)	User on add expense screen	Leave field blank	Message displayed: error
5	Expense (Invalid)	User on add expense screen	Enter non numeric value	Error message: rejected
6	Backend API Success	Backend running	Send valid post request	JSON structured response
7	Backend API Valid Fail	Backend running	Send post request missing fields	Error JSON response
8	JSON Parsing	Backend running	Trigger api call from app	Data parsed and displayed
9	Network Timeout	Backend unreachable	Attempt submit data	Error message

These test cases provided coverage of critical user flows and the system behaviour.

5.5 Minimum Unit Testing

To ensure correct logic, a minimum set of unit tests was added.

Backend Unit Tests Covered

Validation fails when required fields missing

Validation fails fo invalid numeric values

Error responses follow expected JSON structure

Flutter unit/widget tests covered:

Add expense from validation behaviour

Widget rendering for screens

Handling of successful API responses

Handling of failed API responses without crashing the app.

These test helped ensure individual components behave correctly by themselves.

5.6 Test Results Summary

All major functional and unit testing executed correctly. The application successfully process valid financial entries and rejects invalid input with error messages. Frontend to Backend communication verified and the application handled network failures.

Overall the testing process confirmed that AIFC meets its requirements and operates reliably.

6.Installation Guide

6.1 Overview

This section describes the steps required to install, configure and run the application. It consists of a Flutter frontend and API backend service. Both components must be correctly configured for the application to function properly.

6.2 System requirements

Hardware Essentials *(or the laptop will run very hot)*

Minimum 8GB RAM recommended

Dual Core Processor +

At least 2GB free disk space

Software

Flutter SDK

Android Studio

Android emulator/physical phone

Node js and NPM

Git

API tester (optional)

6.3 Installation Steps

Clone the repository from the gitlab
Set up and run the backend, ensure it starts successfully.
Configure API Address.
Set up the Flutter frontend in the terminal, **flutter pub get**, **flutter doctor**
Start the emulator and **flutter run**

The app should now launch.

6.4 Vertification

Installation is successful if

Backend runs errorless

Flutter app launches

The app successfully communicates with the backend

6.5 Troubleshooting

Ensure the backend is running before starting Flutter

Run Flutter doctor if setup fails

Confirm device and server on the same network.

Summary:

Following these steps will result in a working FinanceCoach installation with proper frontend to backend communication